

Overview

A new utility `detectCommunities` is added to `packages/graph-view/src/utils/louvain.ts` (lines 1-47). It performs Louvain community detection on a user-supplied subset of nodes and edges.

Key changes

- **Imports** added at the top: `Graph` from `graphology`, `louvain` from `graphology-communities-louvain`, and `GraphEdge` type from `@novadiff/graph-core/types` (R1-R3).
- **Exported function** `detectCommunities(nodeIds: string[], edges: GraphEdge[]): Map<string, number>` (R17-R20).
- **Graph construction**: creates an undirected, non-multigraph; adds only nodes in `nodeIds` and edges whose both endpoints are in that set; skips self-loops and duplicate edges (R21-R29).
- **Community assignment**: calls `louvain(g)` (returns `Record<string, number>`), maps results, defaults missing nodes to `-1`, then reassigns any `-1` sentinels to unique ids beyond the current maximum to guarantee distinct communities for disconnected nodes (R30-R46).
- **JSDoc** explains the defensive reassignment and future-proofing against library changes (R5-R16).

Impact

- Adds community-detection capability to graph-view without altering existing APIs.
- Graph construction is linear in the number of provided nodes and edges; Louvain runtime dominates.
- Guarantees unique community ids for disconnected nodes, preventing accidental merging.
- Pure function with no side effects or logging.

Risks & follow-ups

- Verify that `graphology-communities-louvain` is installed and compatible.
- Test edge-filtering logic with mixed edge sets to ensure only valid edges are added.
- Confirm that defensive reassignment does not alter expected community ids for connected components; add regression tests.
- Ensure graceful handling of empty `nodeIds` or `edges`; add edge-case tests.